
FACE
Prep

Manacher's Algorithm

What is Manacher's algorithm?

- Manacher's algorithm is used to find the longest palindromic substring in any given string. This algorithm is faster than the brute force approach, as it exploits the idea of a palindrome happening inside another palindrome.
- Manacher's algorithm is designed to find the palindromic substrings with odd lengths only. To use it for even lengths also, we tweak the input string by inserting the character "#" at the beginning and each alternate position after that (changing "abcaac" to "#a#b#c#a#a#c#").
- In the case of an odd length palindrome, the middle character will be a character of the original string, surrounded by "#".

Steps of the Manacher's algorithm are as follows:

1. Create an array or a list (sCharssChars) of length strLenstrLen which is 2 * n + 32∗n+3 (nn being the length of the given string), to modify the given string.
2. Assign the first and last element of sCharssChars to be "@" and "$", respectively.
3. Fill the blank spaces in sCharssChars by characters of the given string and "#" alternatively.
4. Declare variables
    1. Implicating maximum detected length of palindrome substring maxLen = 0maxLen=0
    2. Position from where to start searching start = 0start=0
    3. Highest position of the extreme right character of all detected palindromes maxRight = 0maxRight=0
    4. Center of the detected palindrome center = 0center=0.
5. Create an array or a list pp to record the width of each palindrome about their center, center being the corresponding characters in sCharssChars.

7.Create a for loop iterating from 11 to strLen - 1$strLen-1$, with ii incrementing in each iteration.

8.In each iteration, check if i < maxRight$i<maxRight$, if yes, then assign minimum of maxRight - i$maxRight-i$ and p[2 * center - i]$p[2*center-i]$ to p[i]$p[i]$.

9.Nest a while loop inside the for loop, to count with width along the center, condition being, sChars[i + p[i] + 1]$sChars[i+p[i]+1]$ is equal to sChars[i - p[i] - 1]$sChars[i-p[i]-1]$, if yes, increment p[i]$p[i]$ by 1.

10.To update center, check if i + p[i]$i+p[i]$ is greater than maxRight$maxRight$, if yes, assign center$center$ to be 11, and maxRight$maxRight$ to be i + p[i]$i+p[i]$.

11.For updating the Maximum length detected, check if p[i]$p[i]$ is greater than maxLen$maxLen$, if yes, then assign start$start$ to be (i - p[i] - 1) / 2$(i-p[i]-1)/2$, and maxLen$maxLen$ to be p[i]$p[i]$.

12.Come out of the for loop, and print the substring in the given string, starting from start$start$ and ending at start + maxLen - 1$start+maxLen-1$.

```
import java.util.*;

class Prep
{
   static void findLongestPalindromicString(String text)
   {
      int N = text.length();
      if (N == 0)
         return;
      N = 2 * N + 1; // Position count
      int[] L = new int[N + 1]; // LPS Length Array
      L[0] = 0;
      L[1] = 1;
      int C = 1; // centerPosition
      int R = 2; // centerRightPosition
      int i = 0; // currentRightPosition
      int iMirror; // currentLeftPosition
      int maxLPSLength = 0;
      int maxLPSCenterPosition = 0;
      int start = -1;
      int end = -1;
      int diff = -1;
      for (i = 2; i < N; i++)
      {
iMirror = 2 * C - i;
         L[i] = 0;
         diff = R - i;

if (diff > 0)
            L[i] = Math.min(L[iMirror], diff);

while (((i + L[i]) + 1 < N && (i - L[i]) > 0) &&
                  (((i + L[i] + 1) % 2 == 0) ||
               (text.charAt((i + L[i] + 1) / 2) ==
                text.charAt((i - L[i] - 1) / 2))))
         {
            L[i]++;
         }
```

THANK YOU

FACE Prep